

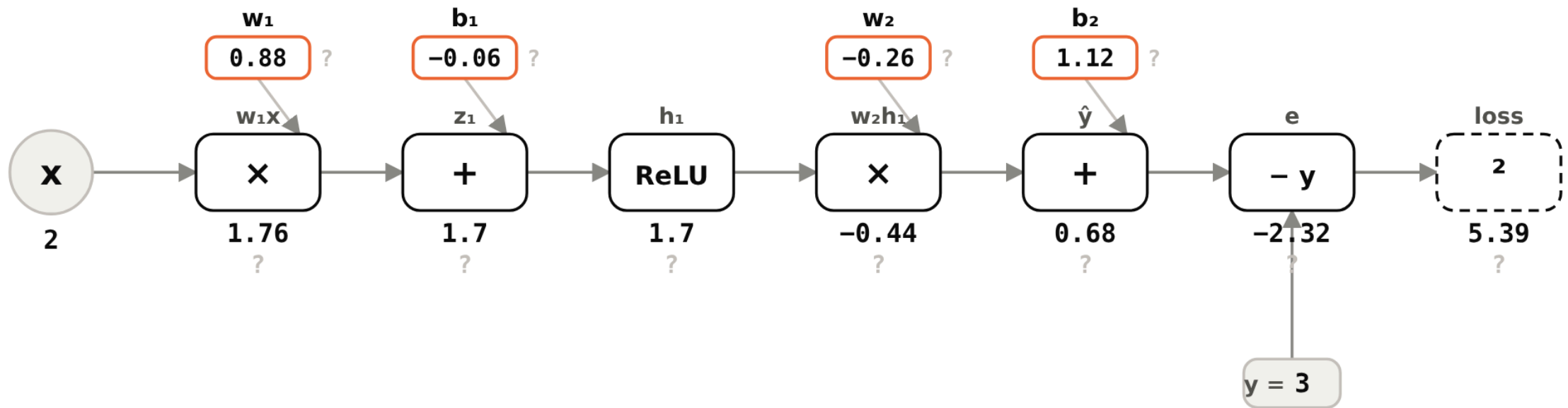
Data & AI

7. Classifying Handwritten Digits

Dasaem Jeong · Art & Technology, Sogang University

Last Time

Backpropagation



- One pass forward and one pass backward give the slope of every parameter
- Run in Neuron Patch is this step, repeated

Today

- What changes when the answer is a category, not a number
- Handwritten digits as 784 numbers in, 10 numbers out
- Your coding agent trains the classifier, and you check what it did

Supervised learning

Every Dataset So Far

Each sample is an input x paired with its answer y

Lecture	Input x	Answer y
3	Floor area of a studio	Monthly rent
3	Study hours	GPA
5	Ratings of three movies	Rating of a fourth movie
6	A point (x, y)	Its color, one of two
Today	784 pixel values	The digit, one of ten

- Supervised learning: the answers are given, and training adjusts the parameters until the model's answers match

Regression and Classification

Regression

- y is a quantity on a number line
- Rent 550,000 won, GPA 3.7
- Loss: $(\hat{y} - y)^2$

Classification

- y is one of a fixed set of categories
- Digit 0 to 9, cat or dog
- Loss: needs a new definition

Labels

The category, written as an integer

Category	Label
The digit "7"	7
Cat, dog, bird	0, 1, 2

- The number is a name, not a quantity: 2 is not twice 1
- The set of labels is fixed before training

Handwritten digits

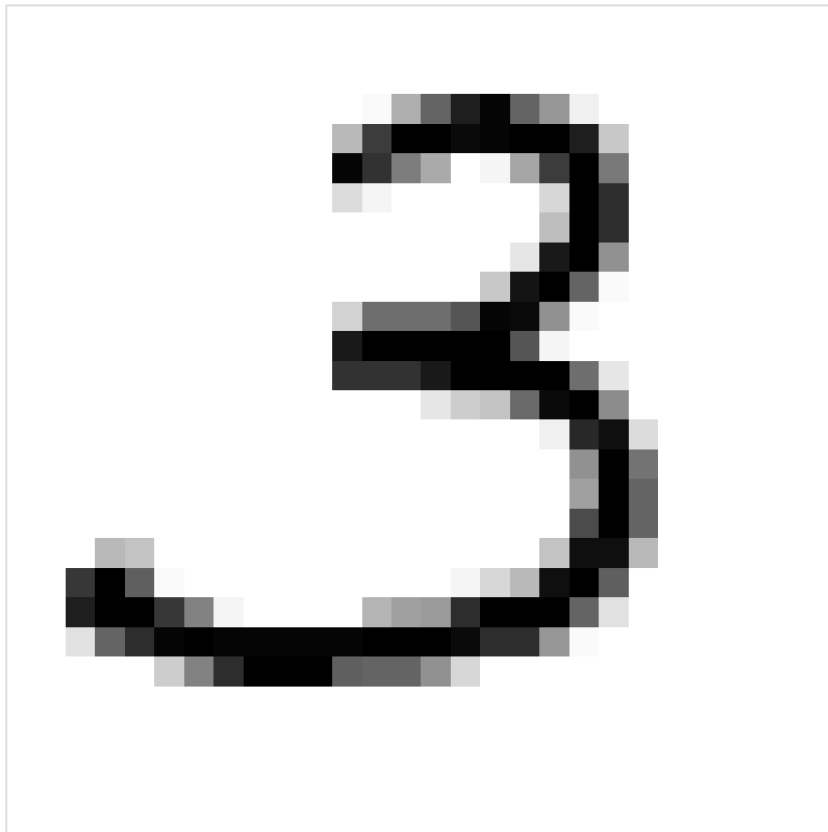
MNIST

70,000 handwritten digits, 28 × 28 pixels

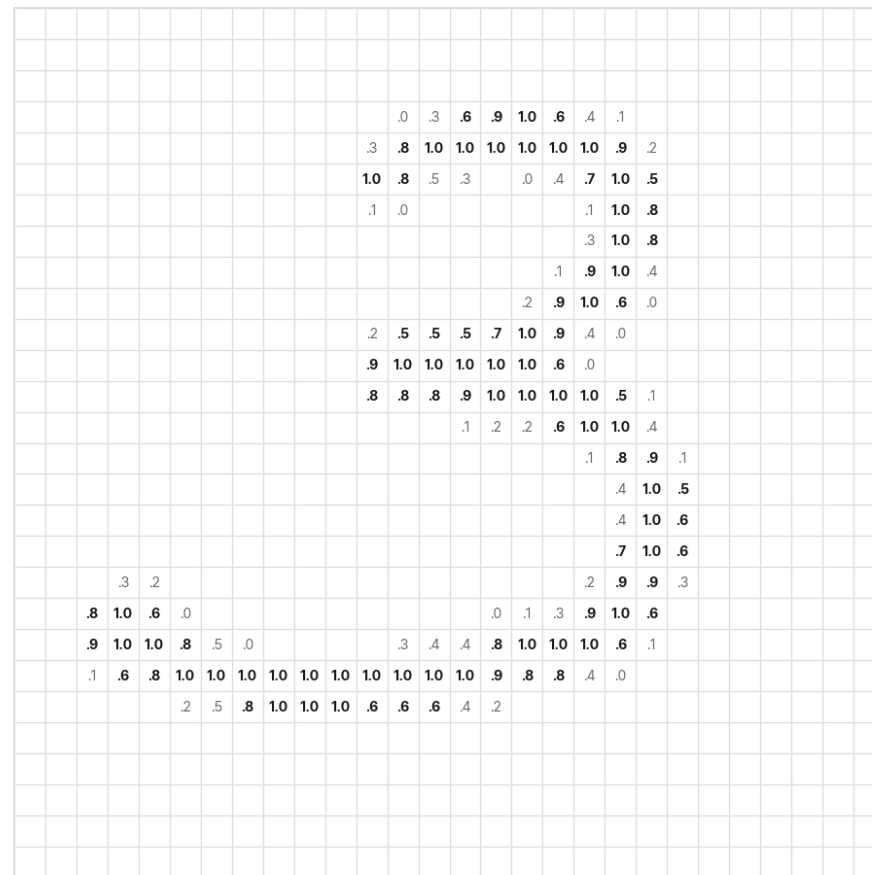


An Image as Numbers

label: 3



$28 \times 28 = 784$ numbers, 0 (blank) to 1



- One pixel is one number between 0 (blank) and 1 (full ink)

Input Vector of 784 Numbers



- The 28 rows laid end to end: the input vector of lecture 5, with 784 entries
- Every image is flattened the same way, so x_1 is always the top left pixel

Ten Outputs

Shapes in the network, as in lecture 5

	Shape	
Input	1×784	one flattened image
Layer 1	$W_1 784 \times 64, b_1 64, \text{ReLU}$	1×64
Layer 2	$W_2 64 \times 10, b_2 10$	1×10

- One output neuron for each digit
- The digit whose neuron scores highest is the answer

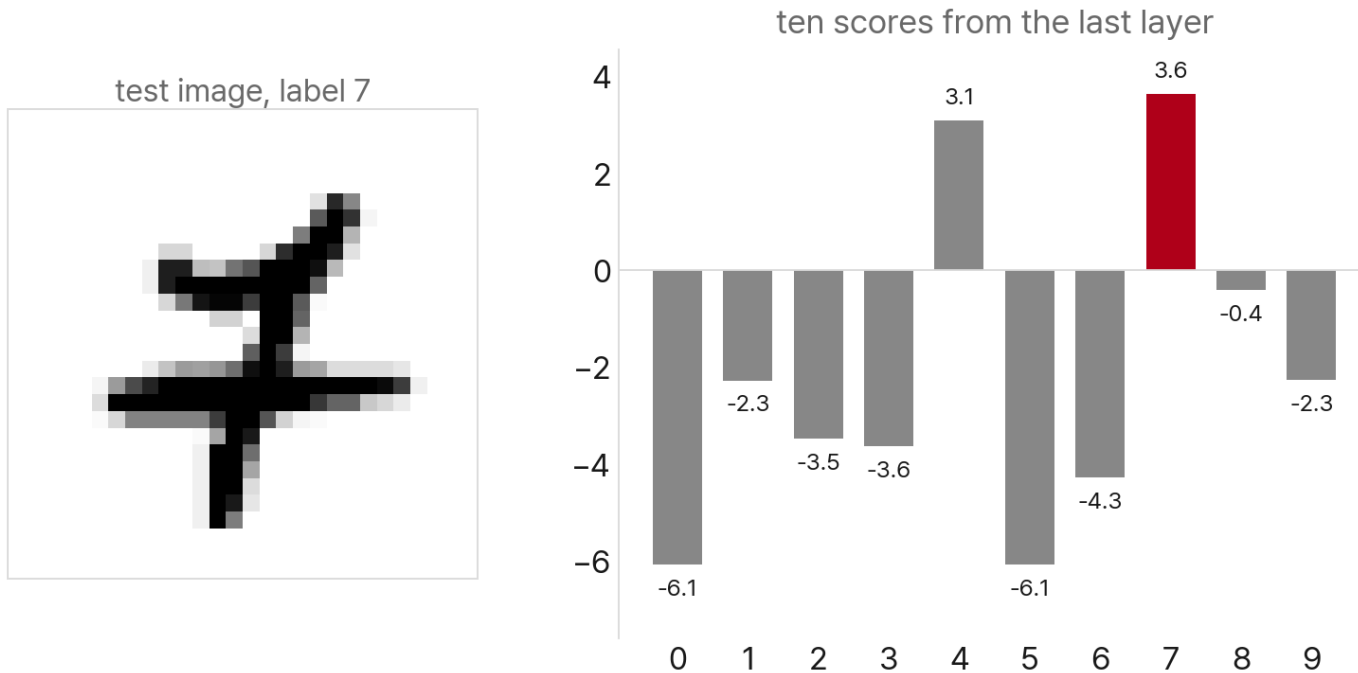
Counting the Parameters

	Count
Layer 1 weights	$784 \times 64 = 50,176$
Layer 1 biases	64
Layer 2 weights	$64 \times 10 = 640$
Layer 2 biases	10
Total	50,890

- The "2 layers" preset in Neuron Patch had 33

Ten Scores

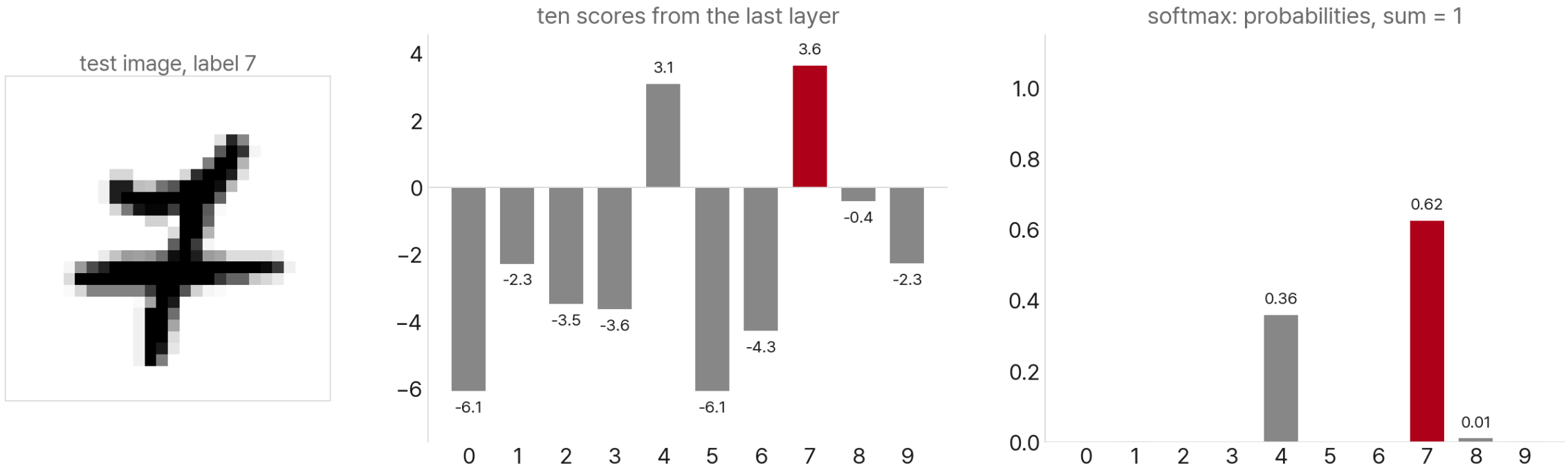
One test image through the trained network



- The last layer has no activation, so scores can be negative
- 7 scores highest, 4 comes second

Softmax

$$p_7 = e^{3.65} \div (e^{-6.08} + e^{-2.27} + \dots + e^{-2.26}) = 0.62$$

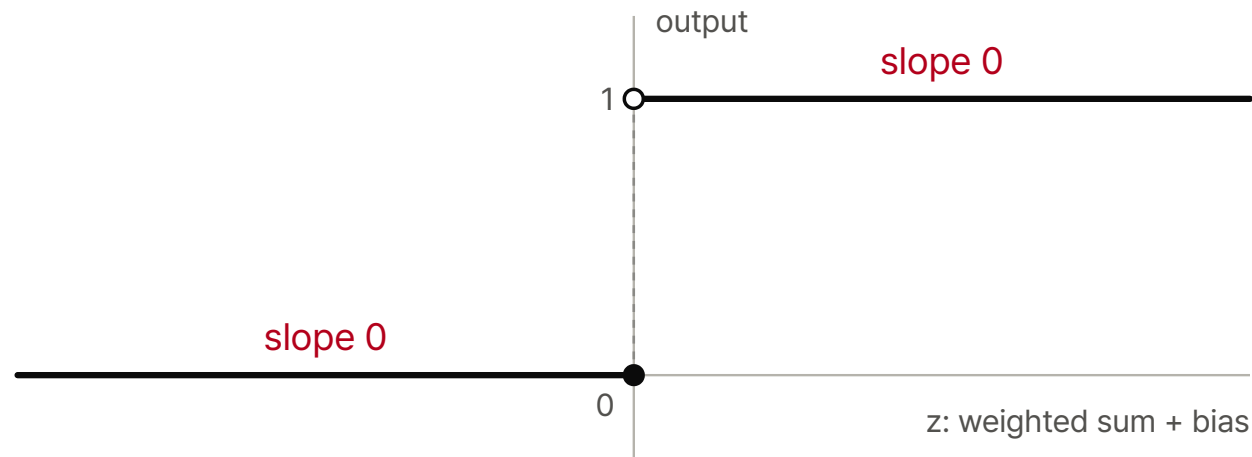


- e^s makes every score positive, and dividing by the sum makes them add up to 1
- The order does not change: 7 is still first, 4 second

A loss for classification

Accuracy Has No Slope

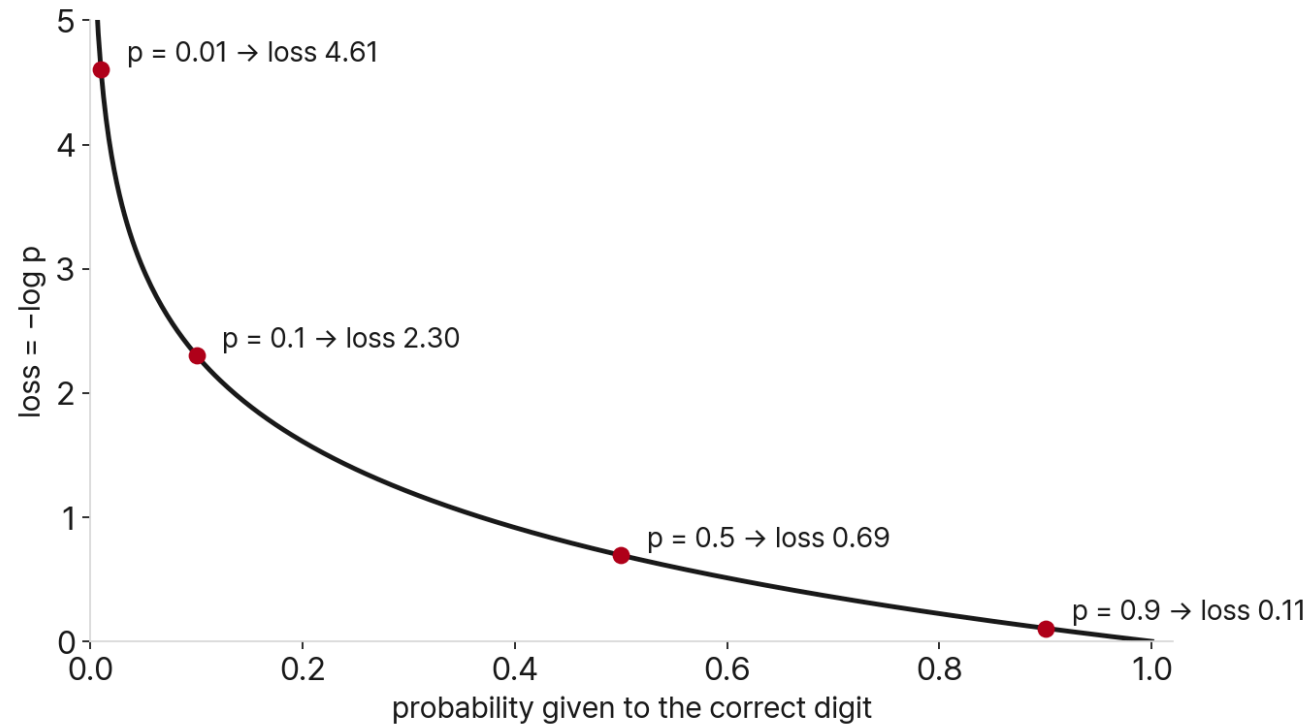
Accuracy: the share of images whose top score is the right digit



- Nudge a weight: the scores move a little, but the top digit almost never changes
- Slope 0 everywhere, like the step function in lecture 5

Cross-Entropy Loss

loss = $-\log p$, where p is the probability the model gave to the correct digit



- Only the probability of the correct digit counts
- Guessing 1/10 for every digit gives 2.30, and the mean over the training images is the loss, as in lecture 6

Three Parts of Machine Learning

- A model: 784 $\rightarrow$ 64 ReLU $\rightarrow$ 10 scores $\rightarrow$ softmax
- A loss: cross-entropy instead of squared error
- A search: gradient descent with backpropagation, unchanged

Training set and test set

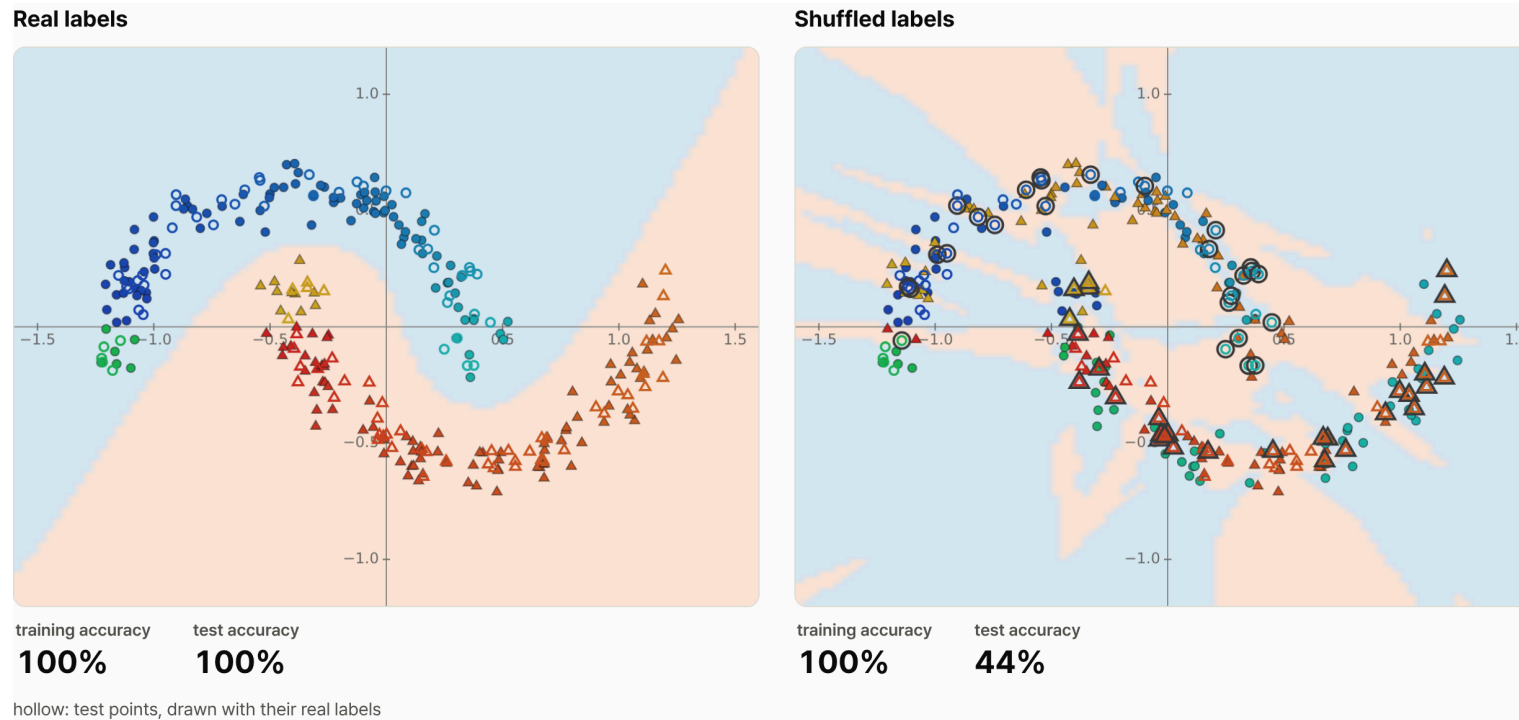
Two Piles of Images

	Training images	Test images
MNIST	60,000	10,000

- Training images set the parameters
- Test images are shown only after training, to measure

Memorizing the Training Points

jdasam.github.io/mas1004/demos/moons-shuffled-labels · width 64



- Same points on both sides, labels shuffled on the right: training accuracy 100% on both
- Test accuracy 100% on the left, 44% on the right

Epochs and Batches

Term	Meaning
Batch	A few hundred images whose mean slope gives one step
Epoch	One pass through all the training images
Example	60,000 images, batches of 200: 300 steps per epoch

- The code your agent writes names both: look for them
- A loss curve usually has one point per epoch

Build it with your agent

Before You Start

Check	How
A new empty folder is open in VS Code	File, Open Folder, name it lecture07
Cline answers	Type hello and wait for a reply

- Python, libraries, data: the agent finds out what is there and sets up the rest
- When it asks to install or run something, read what it wants, then approve

What to Ask For

Say what you want to see, not how to build it

- A model that classifies MNIST images, in Python
- You want to see how the training went and how well the trained model works
- On a local web page where you can check the results yourself
- The agent picks the library, gets the data and chooses the model: your job is to read what it chose

What to Look For

Every agent builds something different. Find these in yours

Question	Where to find it
Where did the data come from, and how many images?	The download lines in the code, the first output lines
What is the model: layers, neurons, parameters?	The lines that build the model
Did the training go down?	The loss per epoch, in the output or on the page
How well does it work, measured on which images?	The accuracy lines: training or test?
What does the page let you check?	Open the HTML file in your browser

- A warning in the output is not an error; if it stops, read the last lines and tell the agent

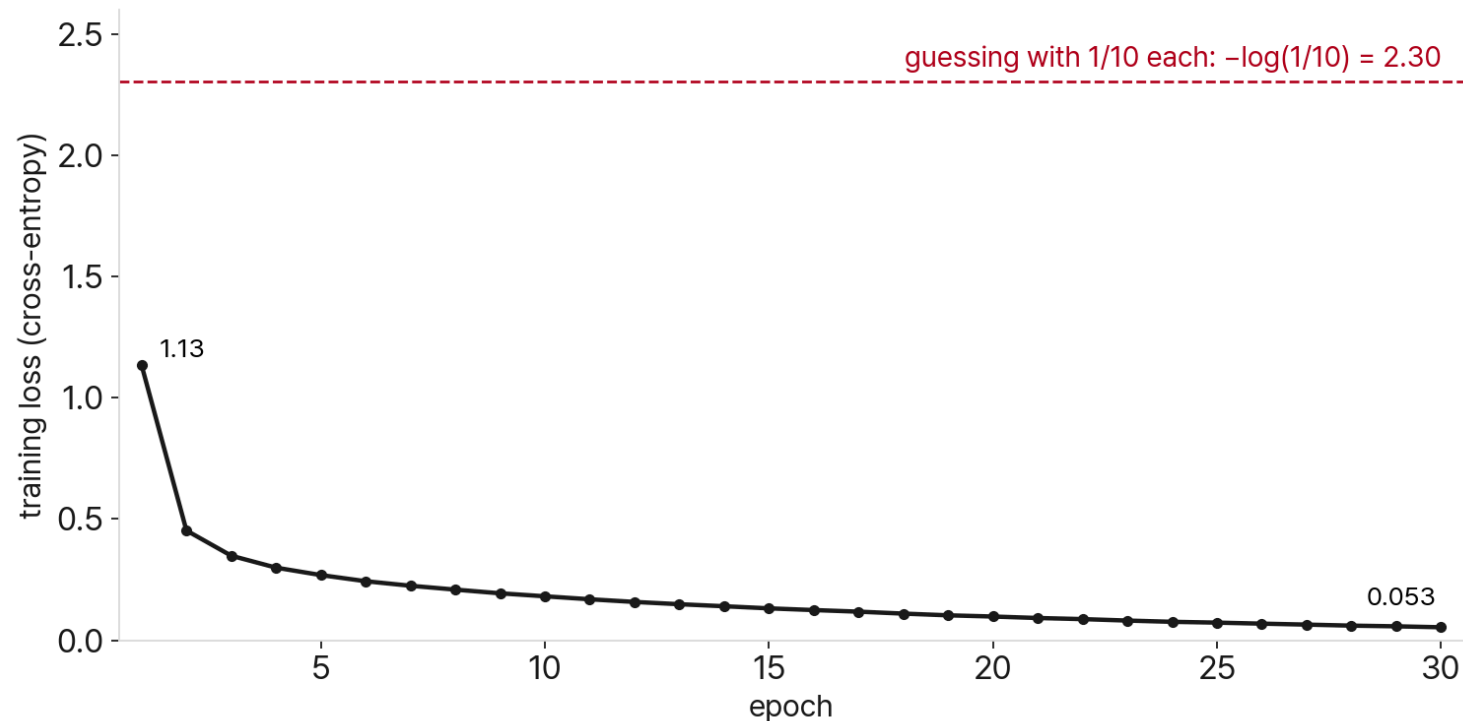
Check Six Things

Answer on the LMS before the end of class. This replaces today's quiz

1. Did the code run to the end? What was the last line of output?
2. How many images went in? Did you look at any of them?
3. How many layers and how many parameters? Find the model in the code, count by hand as on the parameters slide, and compare with what the code prints, if it prints a count
4. Did the loss go down? Read the first and the last value off the curve
5. Which images was each accuracy measured on?
6. Pick one test image on the page: is the prediction right?

Loss Curve

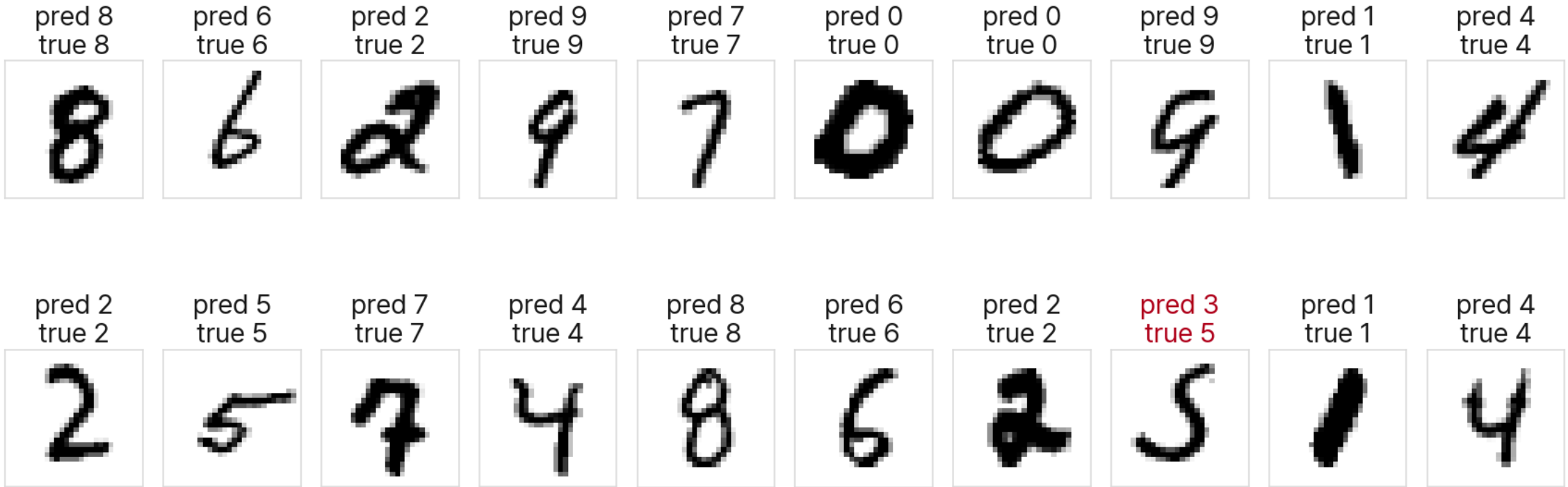
My run: scikit-learn, 10,000 training images, one hidden layer of 64, 30 epochs, 1.3 seconds



- From 1.13 to 0.05
- The first point is the mean over epoch 1, already below the 2.30 of guessing

Twenty Test Images

My run, the first 20 test images



- 19 of 20 right, and a 5 read as a 3
- Test accuracy over all 2,000 images: 94.95%

Two Accuracies

My run, the same model measured twice

Measured on	Accuracy
The 10,000 training images	99.18%
The 2,000 test images	94.95%

- The gap is what the model got right only because it had seen those images
- The number to report is the one on images it never trained on

Confusion Matrix

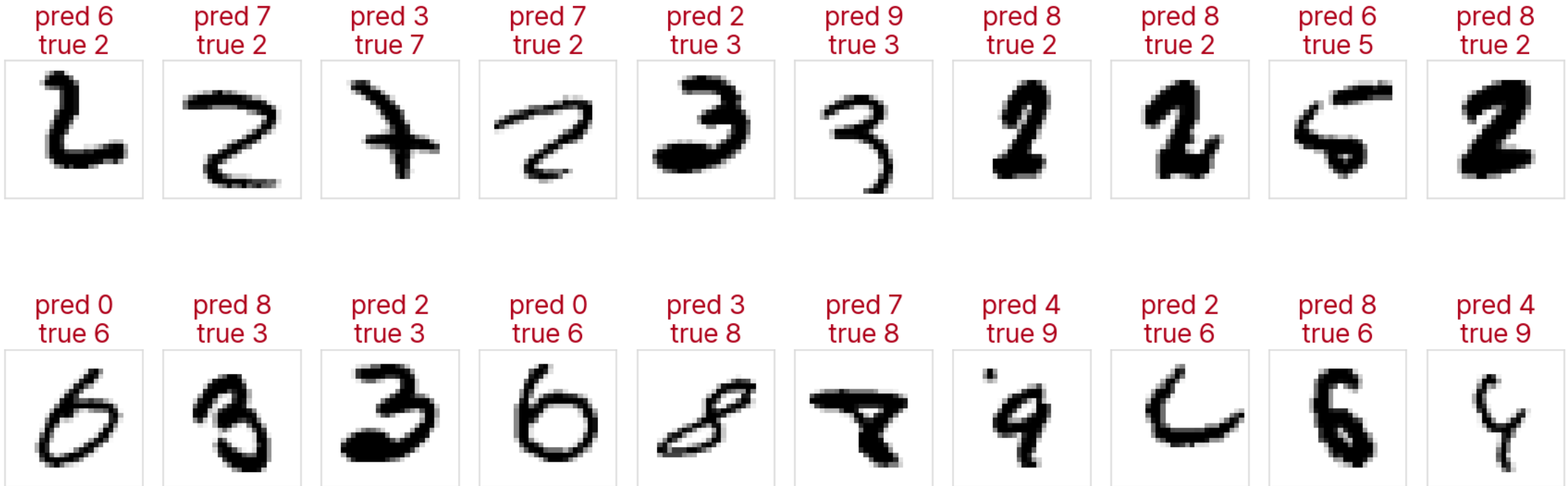
My run: rows are true digits, columns are predicted digits

	predicted digit									
	0	1	2	3	4	5	6	7	8	9
0	186	0	0	1	0	0	2	0	0	0
1	0	245	0	0	0	0	0	0	4	0
2	2	2	207	2	2	0	2	6	4	0
3	0	0	5	172	0	5	0	4	1	1
4	0	0	0	1	186	0	2	0	0	2
5	1	0	0	2	1	164	5	0	3	1
6	3	0	2	0	1	1	189	0	2	0
7	1	0	1	1	1	1	0	189	0	2
8	0	0	1	4	1	2	3	3	167	0
9	0	1	0	2	3	1	0	2	1	194

- 2 read as 7 six times, 3 read as 2 or 5 and 5 read as 6 five times each
- 0 and 1 are read right most often, 2 least often

Confident Mistakes

My run: the 20 wrong predictions with the highest probability



- Would you have read these right?
- How would you ask for these 20 images from your model?

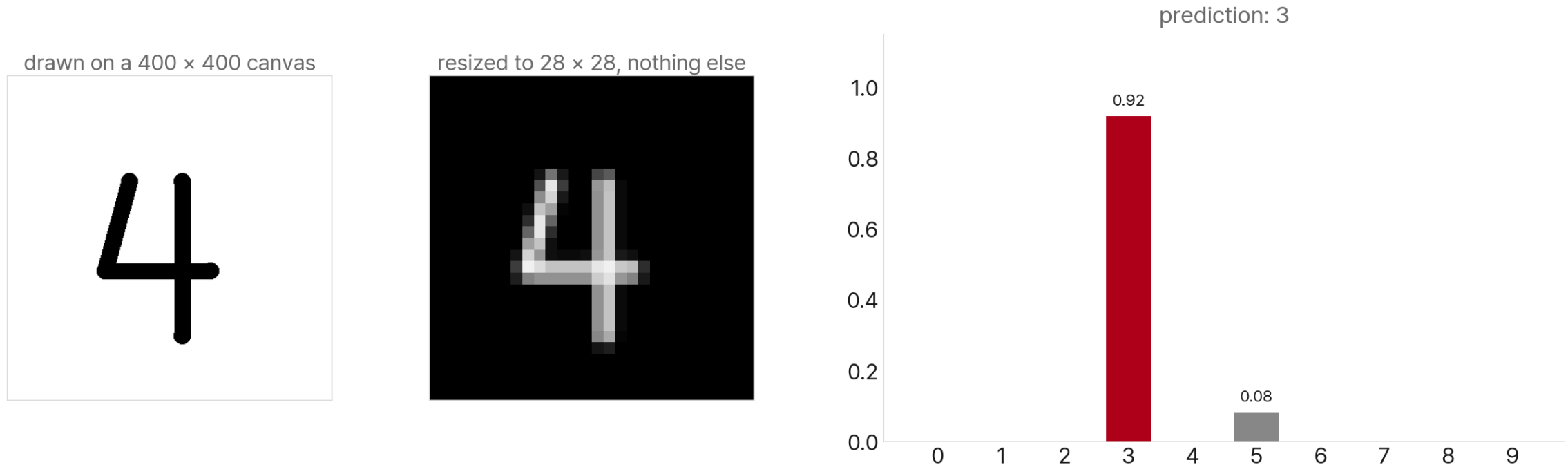
Your own handwriting

Draw It on the Page

- The page should let you draw a digit with the mouse and show what the model reads, with the ten probabilities
- The model has never seen your handwriting: what do you say to your agent to get this?

First Try

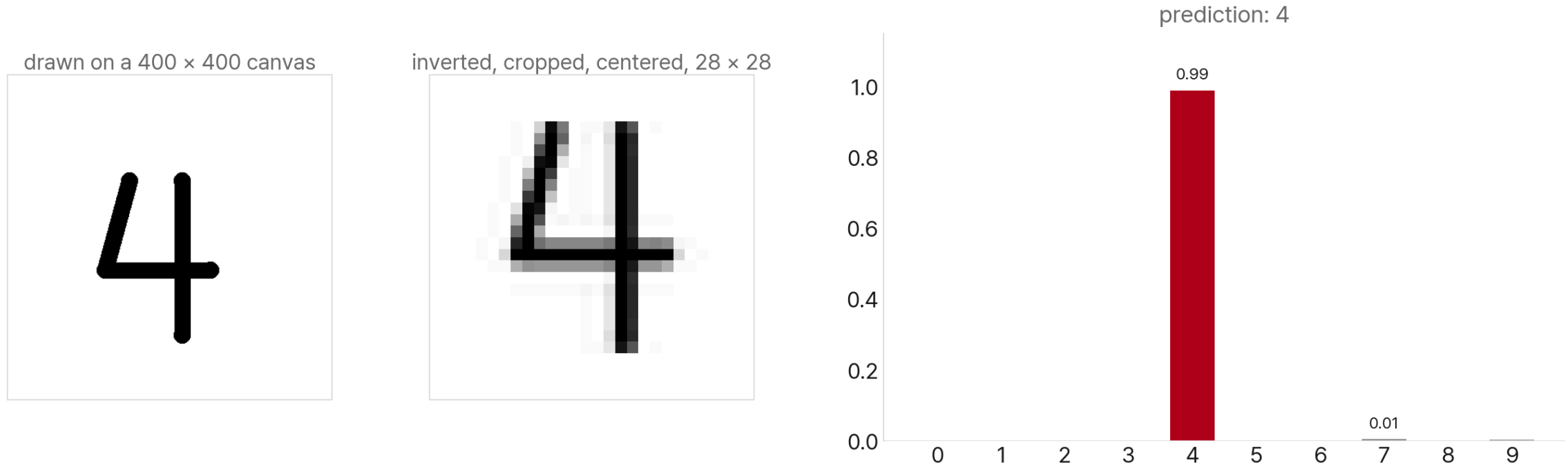
My digit, drawn on a white canvas and shrunk to 28×28 , nothing else



- Predicted 3 with probability 0.92, for a 4
- The training images have 0 for background and 1 for ink: shrunk as it is, the whole white canvas reads as ink

After Preprocessing

Inverted, cropped to the strokes, centered in 28×28



- Predicted 4 with probability 0.99
- Same parameters: only the input was changed to look like the training images

What we have

Assignment 1 Preview

Training an image classifier on your own dataset

- Pick 3 to 5 categories you care about, and collect the images yourself
- The same steps as today: your data, training, test images, then new photos of your own
- Details and the due date after the holiday

Coming Up

- Sep 24: Chuseok, no class
- Sep 29: Assignment 1 in full, and collecting your own images
- Then: the training loop written out in PyTorch, and images with more than one color channel

What else could the same code classify?